

Vergleich von Buchstaben-Trigrammen (skip = 0 und skip = 1) bei Herodot und Thukydides – mit relativen k-Werten (V8, kommentiert)

Charlotte Schubert

2026-04-15

```
library(tidyverse)
library(here)
library(xml2)
library(tidytext)
library(viridis)
library(stringi)
library(openxlsx)
library(sysfonts)
library(showtext)
font_add_google("Noto Sans", "noto")
showtext_auto()
greek_font <- "noto"
```

1. Normalisierungspipeline

```
# Normalisierungspipeline nach stylo-ah-online
normalise_greek <- function(x) {
  x |>
    str_replace_all("([\\p{Greek}])[-\\u2010-\\u2015]\\s*([\\p{Greek}])",
"\\1\\2") |>
    stri_trans_nfkd() |>
    str_replace_all("[\\u2010-\\u2015\\u2212\\uFE58\\uFE63\\uFF0D]", " ") |>
    str_remove_all("[\\p{M}]") |>
    str_remove_all("[†*<#§]") |>
    str_replace_all("\\u03C2", "\\u03C3") |>
    str_remove_all("[()\\[\\]\\{\\}<>\\u27E8\\u27E9\\u2329\\u232A]") |>
    str_remove_all("[\\p{P}]") |>
    str_to_lower() |>
    str_squish()
}
```

2. Hilfsfunktionen

```
import_text_tlg <- function(filepath, book_col = "17") {
  data <- read_xml(filepath) |> xml_ns_strip()
  text <- xml_children(data) |> xml_text() |> str_trim()
```

```

rename_vec <- c(author_id = "l0", work_id = "l1", work_title = "l2")
rename_vec["book"] <- book_col
xml_children(data) |> xml_attrs() |> bind_rows() |>
  mutate(text = text) |> rename(any_of(rename_vec))
}

# Trigramme auf vollem Text (inkl. Leerzeichen)
# skip = 0: Pos. i, i+1, i+2 (kontiguos)
# skip = 1: Pos. i, i+2, i+4 (ein Zeichen übersprungen)
extract_skip_trigrams <- function(text, skip = 0) {
  map(text, \(txt) {
    chars <- stri_split_boundaries(txt, type = "character")[[1]]
    n <- length(chars)
    step <- skip + 1
    max_start <- n - 2 * step
    if (max_start < 1) return(character(0))
    map_chr(seq_len(max_start), \(i) {
      str_c(chars[i], chars[i + step], chars[i + 2 * step])
    })
  })
}

# Export Resultate in Excel
export_results <- function(results, file_name, overwrite = FALSE) {
  sheet_names <- names(results)
  wb <- createWorkbook()
  walk(sheet_names, \(sheet){
    addWorksheet(wb, sheet)
    writeData(wb, sheet = sheet, x = results[[sheet]])
  })
  saveWorkbook(wb, file_name, overwrite)
}

# k-Wert-Berechnung: absolut und relativ
compute_k_values <- function(freq_tbl, group_col = "author",
                             count_col = "n", total_col = "author_total") {
  freq_tbl |>
    group_by(across(all_of(group_col))) |>
    arrange(desc(.data[[count_col]]), .by_group = TRUE) |>
    mutate(
      rank = row_number(),
      cumulative_pct = cumsum(.data[[count_col]]) /
        first(.data[[total_col]]) * 100,
      n_types = n(),
      n_tokens = first(.data[[total_col]])
    ) |>
    filter(cumulative_pct >= 50) |>
    slice_head(n = 1) |>
    ungroup() |>
    mutate(
      k_rel_types = rank / n_types * 100, # in %

```

```

    k_rel_tokens = rank / n_tokens * 1000 # in %
  ) |>
  select(all_of(group_col), k_abs = rank, bei_pct = cumulative_pct,
         n_types, n_tokens, k_rel_types, k_rel_tokens)
}

```

3. Datenimport

```

autoren <- tribble(
  ~author,      ~folder,      ~book_col,
  "Herodot",    "herodot_all", "17",
  "Thukydides", "Thucydides_all", "16"
)

all_works <- autoren |>
  pmap(\(author, folder, book_col) {
    list.files(here(folder), pattern = "\\*.xml$", full.names = TRUE) |>
    map(\(f) import_text_tlg(f, book_col = book_col)) |>
    bind_rows() |> mutate(author = author)
  }) |> bind_rows()

all_books_collapsed <- all_works |>
  select(author, book, text) |>
  group_by(author, book) |>
  summarise(text = str_c(text, collapse = " "), .groups = "drop") |>
  mutate(text = normalise_greek(text))

```

4. Wort-Tokenisierung

```

all_tokens <- all_books_collapsed |>
  unnest_tokens(word, text, to_lower = FALSE)

total_words <- nrow(all_tokens)

word_counts <- all_tokens |>
  count(word, sort = TRUE) |>
  mutate(percentage = n / total_words * 100,
         cumulative_pct = cumsum(n) / total_words * 100,
         rank = row_number())

top_50_words <- word_counts |> slice_head(n = 50)
top_200_words <- word_counts |> slice_head(n = 200)

tokens_per_author <- all_tokens |>
  count(author, word, name = "n") |>
  group_by(author) |>
  mutate(author_total = sum(n), pct = n / author_total * 100) |>
  ungroup()

```

```
tokens_per_book <- all_tokens |>
  count(author, book, word, name = "n") |>
  group_by(author, book) |>
  mutate(book_total = sum(n), pct = n / book_total * 100) |>
  ungroup()
```

5. Trigramme: skip = 0 und skip = 1

```
skip_values <- c(0, 1)

trigram_data <- map(skip_values, \(s) {
  label <- sprintf("skip_%d", s)

  per_book <- all_books_collapsed |>
    mutate(trigrams = extract_skip_trigrams(text, skip = s)) |>
    select(author, book, trigrams) |>
    unnest(trigrams) |>
    rename(trigram = trigrams)

  total <- nrow(per_book)
  tokens_autor <- per_book |> count(author, name = "token_count")

  counts <- per_book |>
    count(trigram, sort = TRUE) |>
    mutate(percentage = n / total * 100,
           cumulative_pct = cumsum(n) / total * 100,
           rank = row_number())

  per_author <- per_book |>
    count(author, trigram, name = "n") |>
    group_by(author) |>
    mutate(author_total = sum(n), pct = n / author_total * 100) |>
    ungroup()

  per_book_counts <- per_book |>
    count(author, book, trigram, name = "n") |>
    group_by(author, book) |>
    mutate(book_total = sum(n), pct = n / book_total * 100) |>
    ungroup()

  k_per_author <- compute_k_values(per_author)

  list(
    skip      = s,
    label     = label,
    per_book  = per_book,
    total     = total,
    tokens_autor = tokens_autor,
    counts    = counts,
```

```

    per_author    = per_author,
    book_counts   = per_book_counts,
    k_per_author  = k_per_author,
    top_50        = counts |> slice_head(n = 50),
    top_200       = counts |> slice_head(n = 200)
  )
}) |>
  set_names(sprintf("skip_%d", skip_values))

s0 <- trigram_data$skip_0
s1 <- trigram_data$skip_1

```

5b. Excel-Export

```

export_results(s0, "Trigramresults_skip0.xlsx", overwrite = TRUE)
export_results(s1, "Trigramresults_skip1.xlsx", overwrite = TRUE)

```

5c. CSV-Export: Trigramm-Token-Liste (skip = 0)

```

dir.create(here("export"), showWarnings = FALSE)

for (a in unique(s0$per_author$author)) {
  data <- s0$per_author |>
    filter(author == a) |>
    arrange(desc(n)) |>
    mutate(rank = row_number()) |>
    select(trigram, n, pct, rank)

  filename <- sprintf("trigramme_skip0_%s.csv", str_to_lower(a))
  write_csv(data, here("export", filename))
  cat(sprintf("Exportiert: %s (%d Types, %d Tokens)\n",
              filename, nrow(data), sum(data$n)))
}

Exportiert: trigramme_skip0_herodot.csv (4959 Types, 1150334 Tokens)
Exportiert: trigramme_skip0_thukydides.csv (4429 Types, 947511 Tokens)

```

6. Wort-k-Werte

```

k_words_global <- word_counts |>
  filter(cumulative_pct >= 50) |> slice_head(n = 1)

k_words_per_author <- compute_k_values(tokens_per_author)

author_colors <- c("Herodot" = "#2c3e50", "Thukydides" = "#e74c3c")
skip_colors   <- c("skip = 0" = "#8e44ad", "skip = 1" = "#e67e22")

# Ausgabeordner für alle Visualisierungen
output_dir <- here("Output_V8")

```

```
dir.create(output_dir, showWarnings = FALSE)

theme_greek <- function(base_size = 14) {
  theme_minimal(base_size = base_size) %+replace%
  theme(text = element_text(family = greek_font))
}
```

7. Visualisierungen

Die folgenden Visualisierungen dienen dazu, die Häufigkeitsverteilungen auf Wort- und Trigramm-Ebene sowohl deskriptiv darzustellen als auch die stilistischen Unterschiede zwischen Herodot und Thukydides quantitativ sichtbar zu machen. Alle Plots werden im Ordner Output_V8 gesichert.

A1. Top 50 Wörter pro Autor (Facetten)

Zeigt für jedes der 50 häufigsten Wörter im Gesamtkorpus den relativen Anteil (%) bei jedem Autor nebeneinander. `facet_wrap()` erzeugt ein Panelgitter; `semi_join()` stellt sicher, dass nur Wörter gezeigt werden, die in der Top-50-Liste stehen; `scales = "free_x"` erlaubt unterschiedliche Skalen, da die Anteile zwischen `kai` (~5 %) und selteneren Wörtern stark variieren.

```
p_facets_words <- tokens_per_author |>
  semi_join(top_50_words, by = "word") |>
  ggplot(aes(pct, author, fill = author)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = author_colors) +
  facet_wrap(~word, ncol = 5, scales = "free_x") +
  theme_greek(base_size = 9) +
  labs(x = "Anteil (%)", y = NULL,
       title = "Die 50 häufigsten Wörter: Herodot vs. Thukydides")

p_facets_words
ggsave(here("Output_V8", "hdt_thuc_top50_woerter_facets.png"),
       p_facets_words, width = 12, height = 14, dpi = 300)
```



Figure 1

A2. Kumulative Abdeckung (Wörter) mit k-Wert

Zeigt, wie schnell die häufigsten Wörter den Gesamttext abdecken.

`geom_hline(yintercept = 50)` markiert die 50 %-Schwelle; `geom_vline()` markiert den k-Punkt (Rang bei 50 %). Ein niedrigerer k-Punkt = lexikalisch konzentrierter Stil.

```
cumul_per_author_words <- tokens_per_author |>
  group_by(author) |> arrange(desc(n), .by_group = TRUE) |>
  mutate(rank = row_number(),
         cumulative_pct = cumsum(n) / first(author_total) * 100) |>
```

```

filter(cumulative_pct < 52) |> ungroup()

p_cumul_words <- ggplot(cumul_per_author_words, aes(rank, cumulative_pct,
colour = author)) +
  geom_line(linewidth = 1.0) +
  geom_hline(yintercept = 50, linetype = "dashed", color = "grey40") +
  geom_vline(aes(xintercept = rank), data = cumul_per_author_words |>
    filter(cumulative_pct / 50 > 1) |>
    group_by(author) |>
    slice_min(rank, n = 1), linetype = "dashed") +
  geom_label(
    aes(x = k_abs + 2, y = bei_pct + 2, label = k_abs, color = author),
    data = k_words_per_author, show.legend = FALSE) +
  facet_wrap(~author, nrow = 1, scales = "free_x") +
  scale_color_manual(values = author_colors, name = "Autor") +
  labs(x = "Anzahl der Wörter nach Rang", y = "Kumulative Abdeckung (%)",
    title = "Kumulative Textabdeckung (Wörter) mit k-Wert") +
  theme_greek()

p_cumul_words
ggsave(here("Output_V8", "hdt_thuc_kumulative_woerter_kval.png"),
  p_cumul_words, width = 8, height = 5, dpi = 300)

```

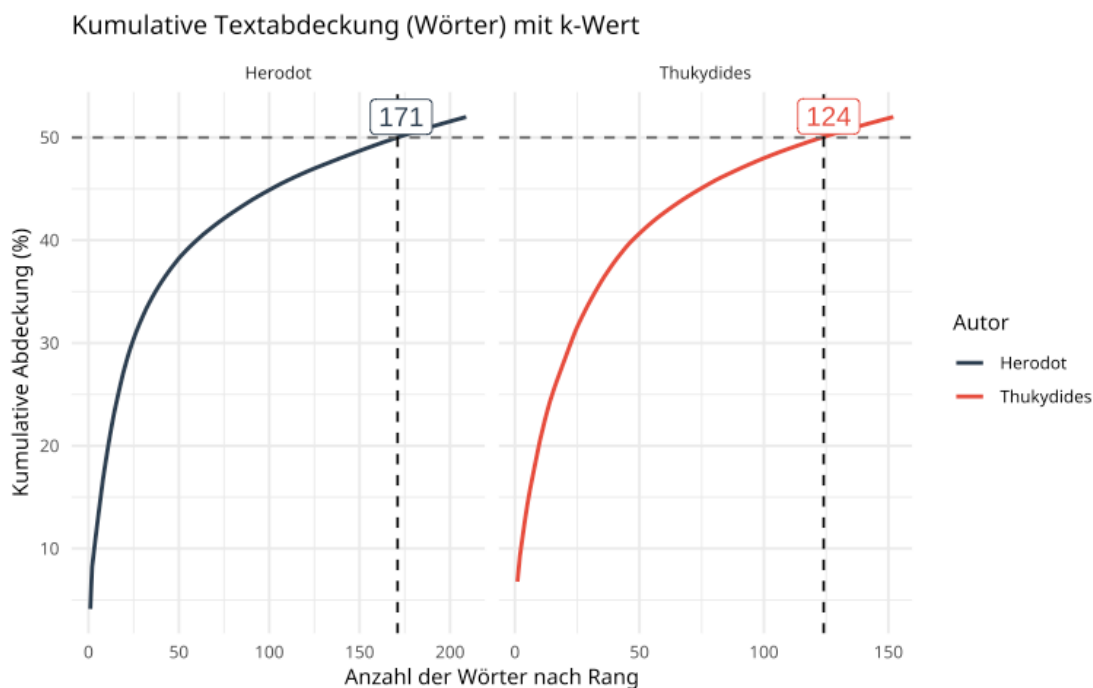


Figure 2

A3. Top 50 Wörter (Balken)

Klassisches Rangfolge-Diagramm. `fct_reorder()` sortiert nach Häufigkeit; horizontale Balken, weil griechische Labels vertikal schlecht lesbar sind.

```

p_bar50_words <- word_counts |> slice_head(n = 50) |>
  mutate(word = fct_reorder(word, percentage)) |>
  ggplot(aes(percentage, word)) +
  geom_col(fill = "steelblue") +
  labs(x = "Anteil am Gesamttext (%)", y = NULL,
       title = "Die 50 häufigsten Wörter (Herodot + Thukydides)") +
  theme_greek() + theme(panel.grid.major.y = element_blank())

p_bar50_words
ggsave(here("Output_V8", "hdt_thuc_top50_woerter_bar.png"),
       p_bar50_words, width = 8, height = 10, dpi = 300)

```

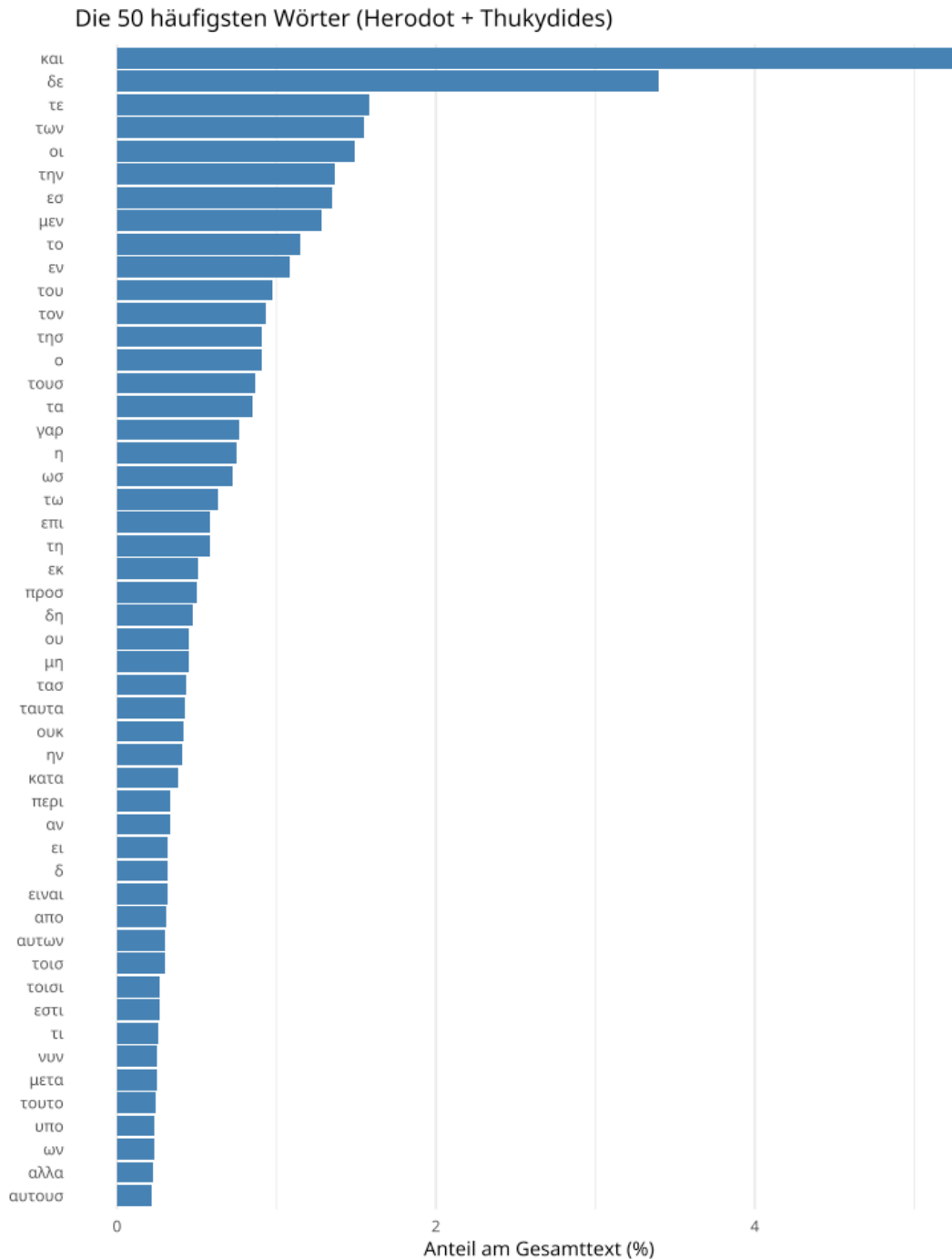


Figure 3

A4. Heatmap: Top 20 Wörter × Autor

`geom_tile()` erzeugt eine Farbmatrix; `scale_fill_viridis()` ist wahrnehmungslinier und farbenblindfreundlich; `geom_text()` zeigt die exakten Prozentwerte in den Kacheln.

```
top_20_words <- word_counts |> slice_head(n = 20) |> pull(word)
```

```
p_heat_words <- tokens_per_author |>
  filter(word %in% top_20_words) |>
  mutate(word = fct_relevel(word, top_20_words)) |>
  ggplot(aes(author, word, fill = pct, label = round(pct, digits = 3))) +
  geom_tile(col = "white") +
  geom_text(col = "white", size = 2) +
  scale_fill_viridis(option = "D", name = "Anteil (%)") +
  labs(x = NULL, y = NULL,
       title = "Die 20 häufigsten Wörter: Herodot vs. Thukydides") +
  theme_greek() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1))

p_heat_words
ggsave(here("Output_V8", "hdt_thuc_heatmap_woerter.png"),
       p_heat_words, width = 10, height = 4, dpi = 300)
```

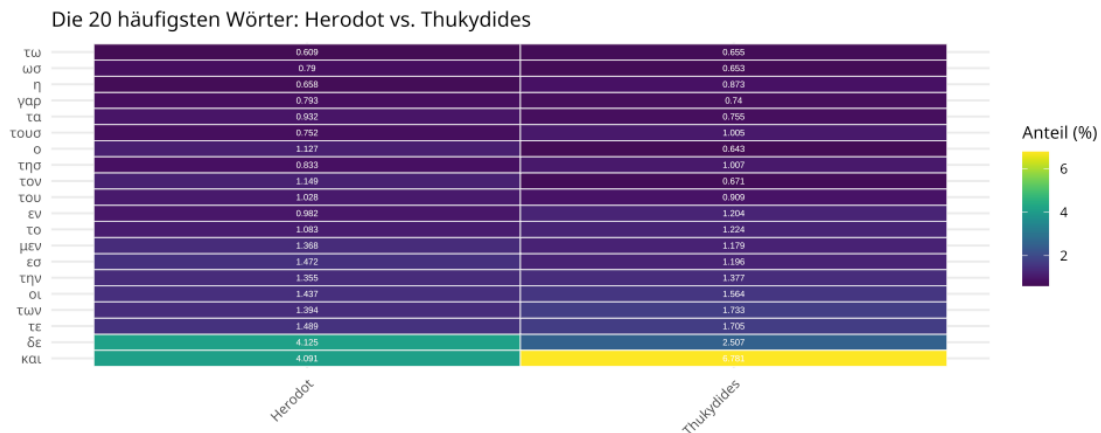


Figure 4

A5. Zipf-Verteilung (Wörter)

Das Zipf-Gesetz besagt: $f(r) \propto 1/r^\alpha$ ($\alpha \approx 1$). In der Log-Log-Darstellung wird daraus eine Gerade mit Steigung $-\alpha$. `geom_smooth(method = "lm")` berechnet die empirische Steigung (= Zipf-Exponent). `geom_abline(intercept = 4, slope = -1)` zeichnet die theoretische Idealgerade als Referenz (Steigung -1 ; Achsenabschnitt $4 \approx \log_{10}$ der maximalen Häufigkeit).

```
zipf_words_author <- tokens_per_author |>
  group_by(author) |> arrange(desc(n), .by_group = TRUE) |>
  mutate(rank = row_number()) |> ungroup()

p_zipf_words <- ggplot(zipf_words_author,
                      aes(log10(rank), log10(n), color = author)) +
  geom_point(alpha = 0.15, size = 0.6) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.8) +
  geom_abline(intercept = 4, slope = -1, linetype = "dashed") +
  scale_color_manual(values = author_colors, name = "Autor") +
```

```
labs(x = expression(log[10](Rang)), y = expression(log[10](Häufigkeit)),
     title = "Zipf-Verteilung der Wörter") +
theme_greek()
```

```
p_zipf_words
ggsave(here("Output_V8", "hdt_thuc_zipf_woerter.png"),
       p_zipf_words, width = 8, height = 5, dpi = 300)
```

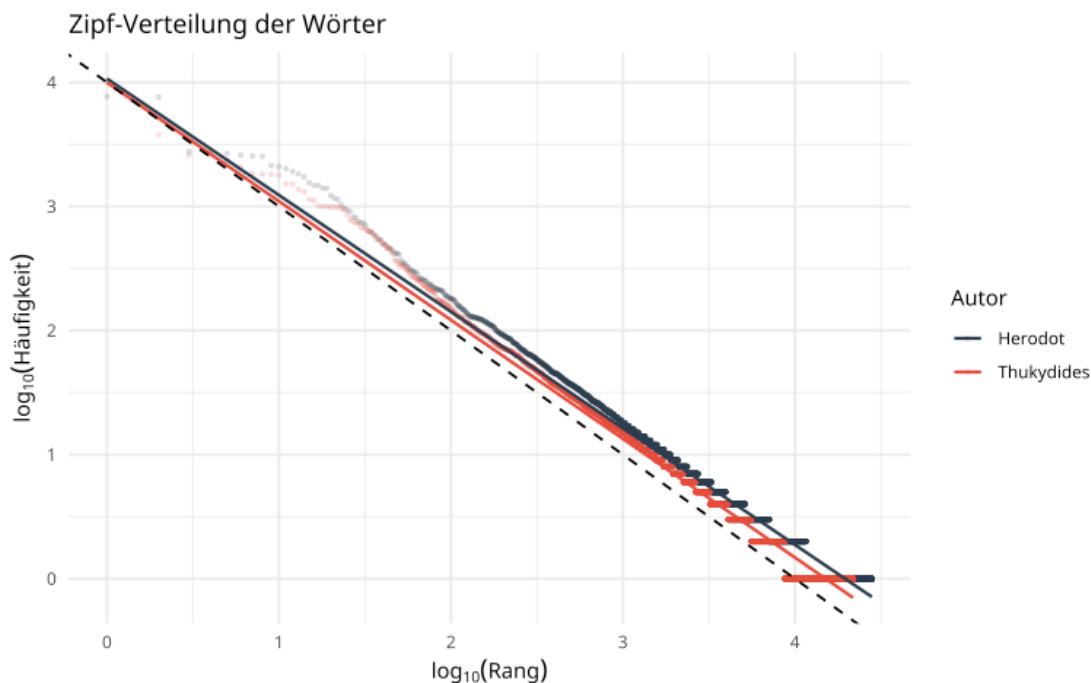


Figure 5

```
zipf_exp_words <- zipf_words_author |>
  group_by(author) |>
  summarise(zipf_exponent = coef(lm(log10(n) ~ log10(rank)))[2],
            .groups = "drop")
knitr::kable(zipf_exp_words, digits = 3, col.names = c("Autor", "Zipf-Exponent"))
```

Autor	Zipf-Exponent
Herodot	-0.939
Thukydidies	-0.957

B. Trigramm-Plots (skip = 0 und skip = 1)

Analog zu A1–A5, aber auf Buchstaben-Trigrammen. Trigramme erfassen morphologische und syntaktische Muster auf Zeichenebene und sind robuster gegenüber Flexionsvariation als Einzelwörter.

```
for (sd in trigram_data) {
  s <- sd$skip
```

```

s_lab <- sprintf("skip = %d", s)
s_tag <- sprintf("skip%d", s)
tri_col <- if (s == 0) "#8e44ad" else "#e67e22"

# B1. Top 50 Trigramme pro Autor (Facetten)
# Trigramme mit Leerzeichen (z.B. "αλ ", " κα") erfassen
# Wortgrenzen-Übergänge und damit syntaktische Kontexte.
p <- sd$per_author |>
  filter(trigram %in% sd$top_50$trigram) |>
  ggplot(aes(pct, author, fill = author)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = author_colors) +
  facet_wrap(~trigram, ncol = 5, scales = "free_x") +
  theme_greek(base_size = 9) +
  labs(x = "Anteil (%)", y = NULL,
        title = sprintf("Top 50 Trigramme (%s): Herodot vs. Thukydides",
s_lab))
print(p)
ggsave(here("Output_V8", sprintf("hdt_thuc_top50_tri_%s_facets.png",
s_tag)),
        p, width = 12, height = 14, dpi = 300)

# B2. Top 50 Trigramme (Balken)
# Farbe: lila (skip=0) vs. orange (skip=1) zur visuellen Zuordnung.
p <- sd$top_50 |>
  mutate(trigram = fct_reorder(trigram, percentage)) |>
  ggplot(aes(percentage, trigram)) +
  geom_col(fill = tri_col) +
  labs(x = "Anteil an allen Trigrammen (%)", y = NULL,
        title = sprintf("Top 50 Trigramme (%s)", s_lab)) +
  theme_greek(base_size = 12) +
  theme(panel.grid.major.y = element_blank())
print(p)
ggsave(here("Output_V8", sprintf("hdt_thuc_top50_tri_%s_bar.png", s_tag)),
        p, width = 8, height = 12, dpi = 300)

# B3. Kumulative Abdeckung mit k-Wert
# Zentraler Messwert: k-Punkt auf Trigramm-Ebene.
# Raute (shape=18) markiert den k-Punkt.
cumul <- sd$per_author |>
  group_by(author) |> arrange(desc(n), .by_group = TRUE) |>
  mutate(rank = row_number(),
         cumulative_pct = cumsum(n) / first(author_total) * 100) |>
  filter(rank <= 200) |> ungroup()

p <- ggplot(cumul, aes(rank, cumulative_pct, color = author)) +
  geom_line(linewidth = 1.0) +
  geom_hline(yintercept = 50, linetype = "dashed", color = "grey40") +
  geom_point(data = sd$k_per_author |>
    rename(rank = k_abs, cumulative_pct = bei_pct),
            size = 4, shape = 18) +

```

```

    geom_text(data = sd$k_per_author,
              aes(x = k_abs + 5, y = bei_pct + 2,
                  label = sprintf("k = %d", k_abs)),
              hjust = 0, size = 4, show.legend = FALSE, family = greek_font)
+
    scale_color_manual(values = author_colors, name = "Autor") +
    labs(x = "Rang (1-200)", y = "Kumulative Abdeckung (%)",
         title = sprintf("Kumulative Abdeckung Trigramme (%s) mit k-Wert",
s_lab)) +
    theme_greek()
print(p)
ggsave(here("Output_V8", sprintf("hdt_thuc_kumul_tri_%s_kval.png", s_tag)),
       p, width = 8, height = 5, dpi = 300)

# B4. Heatmap Top 20 Trigramme x Autor
# Farbunterschiede = autorenspezifische morphologische/syntaktische Muster.
top_20 <- sd$counts |> slice_head(n = 20) |> pull(trigram)
p <- sd$per_author |>
  filter(trigram %in% top_20) |>
  mutate(trigram = fct_relevel(trigram, top_20)) |>
  ggplot(aes(trigram, author, fill = pct)) + geom_tile() +
  scale_fill_viridis(option = "C", name = "Anteil (%)") +
  labs(x = NULL, y = NULL,
       title = sprintf("Top 20 Trigramme (%s): Herodot vs. Thukydides",
s_lab)) +
  theme_greek() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1))
print(p)
ggsave(here("Output_V8", sprintf("hdt_thuc_heatmap_tri_%s.png", s_tag)),
       p, width = 10, height = 4, dpi = 300)

# B5. Zipf (Trigramme)
# Trigramme fallen steiler ab ( $\approx -1.8$  statt  $\approx -0.9$ ), weil weniger
# Types existieren, die häufigsten aber stärker dominieren.
zipf <- sd$per_author |>
  group_by(author) |> arrange(desc(n), .by_group = TRUE) |>
  mutate(rank = row_number()) |> ungroup()
p <- ggplot(zipf, aes(log10(rank), log10(n), color = author)) +
  geom_point(alpha = 0.15, size = 0.6) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.8) +
  scale_color_manual(values = author_colors, name = "Autor") +
  labs(x = expression(log[10](Rang)), y = expression(log[10](Häufigkeit)),
       title = sprintf("Zipf-Verteilung Trigramme (%s)", s_lab)) +
  theme_greek()
print(p)
ggsave(here("Output_V8", sprintf("hdt_thuc_zipf_tri_%s.png", s_tag)),
       p, width = 8, height = 5, dpi = 300)

# B6. Verlauf über Bücher (Top 6 Trigramme)
# Prüft Stabilität über Bücher: gattungstypisch vs. autorenspezifisch.
# shape = 16/17 (Kreis/Dreieck) für S/W-Unterscheidbarkeit.

```

```

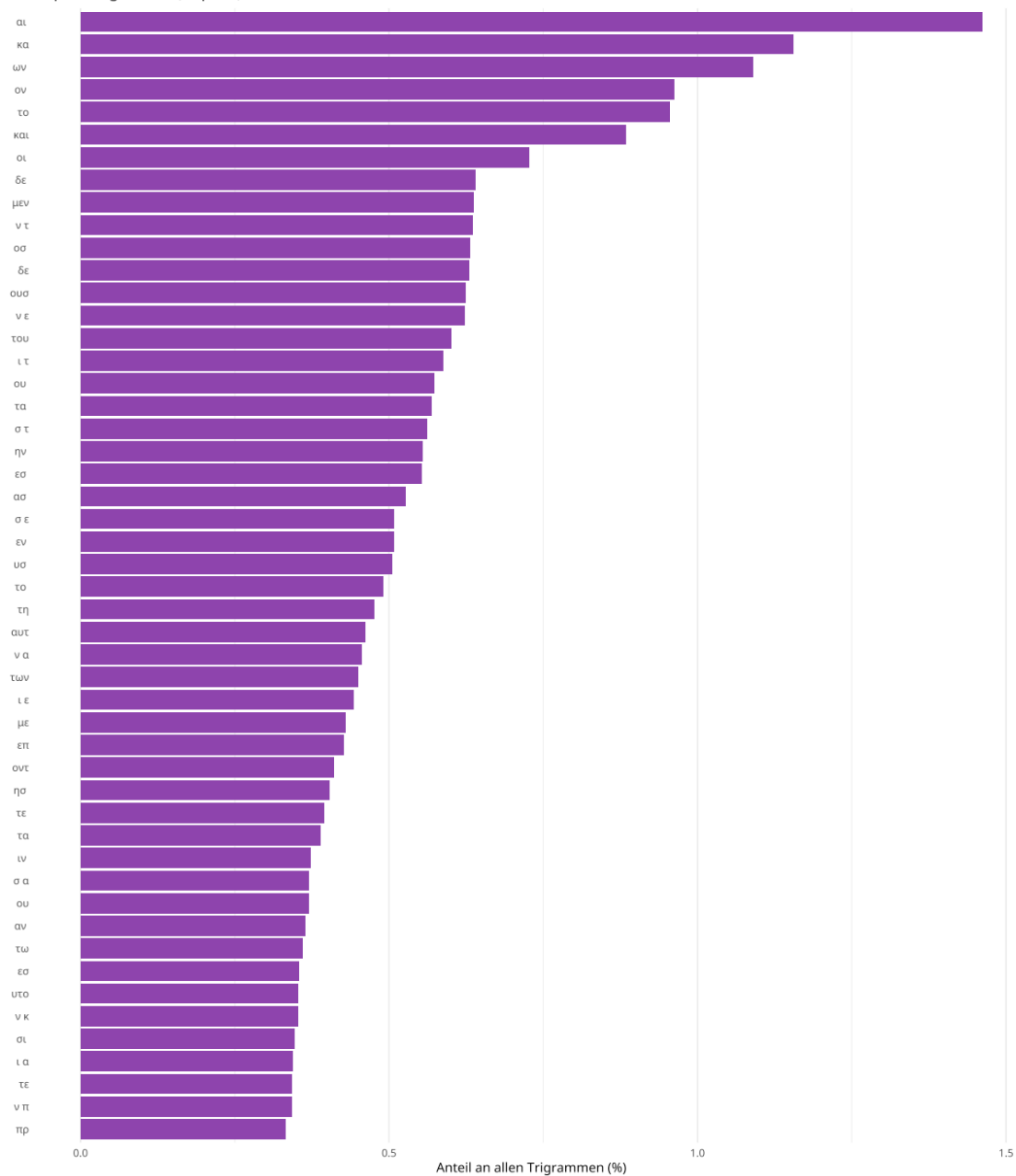
focus <- sd$counts |> slice_head(n = 6) |> pull(trigram)
p <- sd$book_counts |>
  filter(trigram %in% focus) |>
  mutate(book_nr = parse_number(book)) |>
  ggplot(aes(book_nr, pct, color = author, shape = author)) +
  geom_line(linewidth = 0.8) + geom_point(size = 2.5) +
  scale_color_manual(values = author_colors, name = "Autor") +
  scale_shape_manual(values = c("Herodot" = 16, "Thukydides" = 17),
    name = "Autor") +
  scale_x_continuous(breaks = 1:9) +
  facet_wrap(~trigram, scales = "free_y", ncol = 2) +
  labs(x = "Buch", y = "Anteil am Buchtext (%)",
    title = sprintf("Ausgewählte Trigramme (%s): Verlauf über die
Bücher",
                      s_lab)) +
  theme_greek(base_size = 12) +
  theme(strip.text = element_text(size = 12, face = "bold"),
    legend.position = "bottom")
print(p)
ggsave(here("Output_V8", sprintf("hdt_thuc_tri_%s_verlauf.png", s_tag)),
  p, width = 10, height = 8, dpi = 300)
}

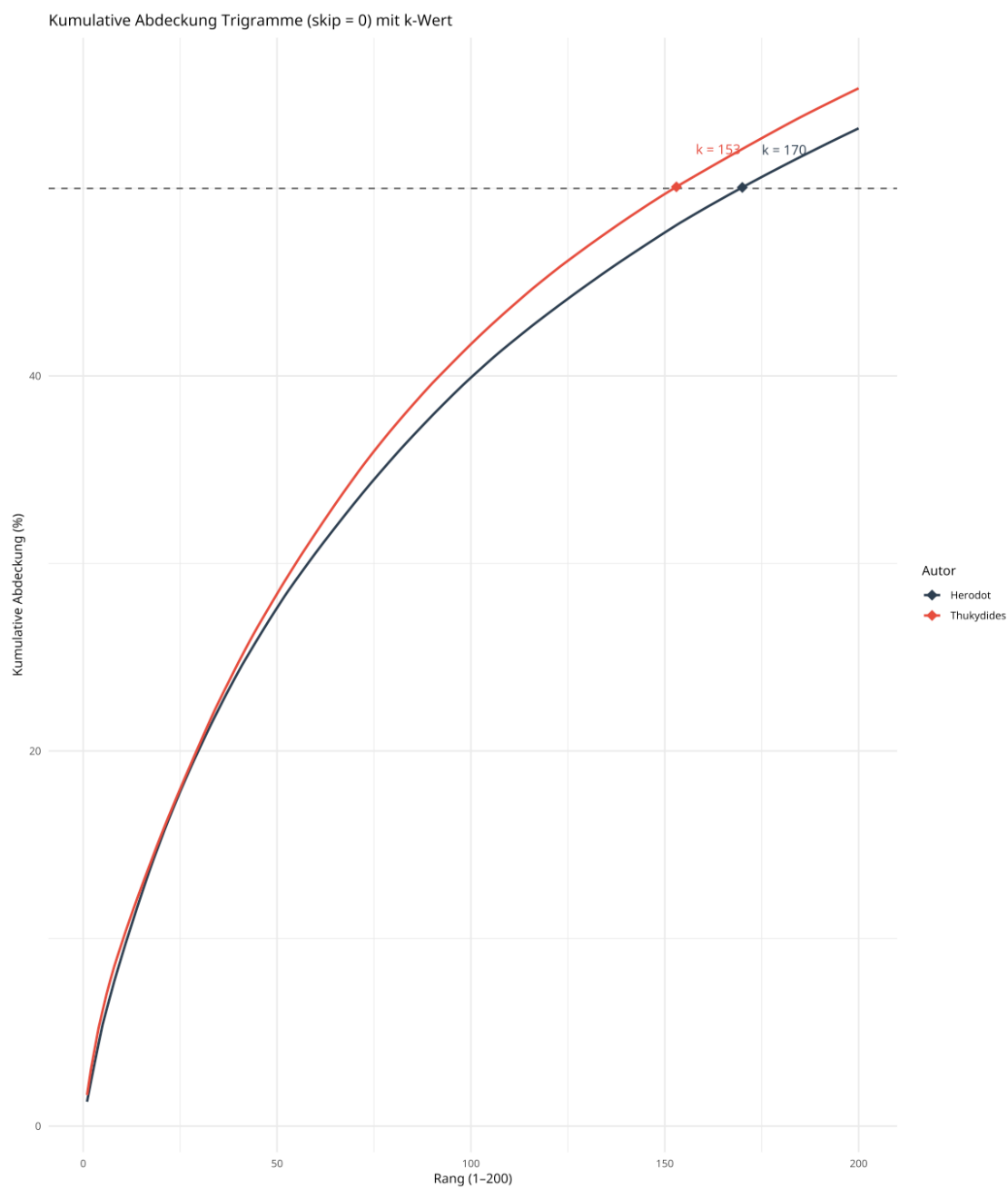
```

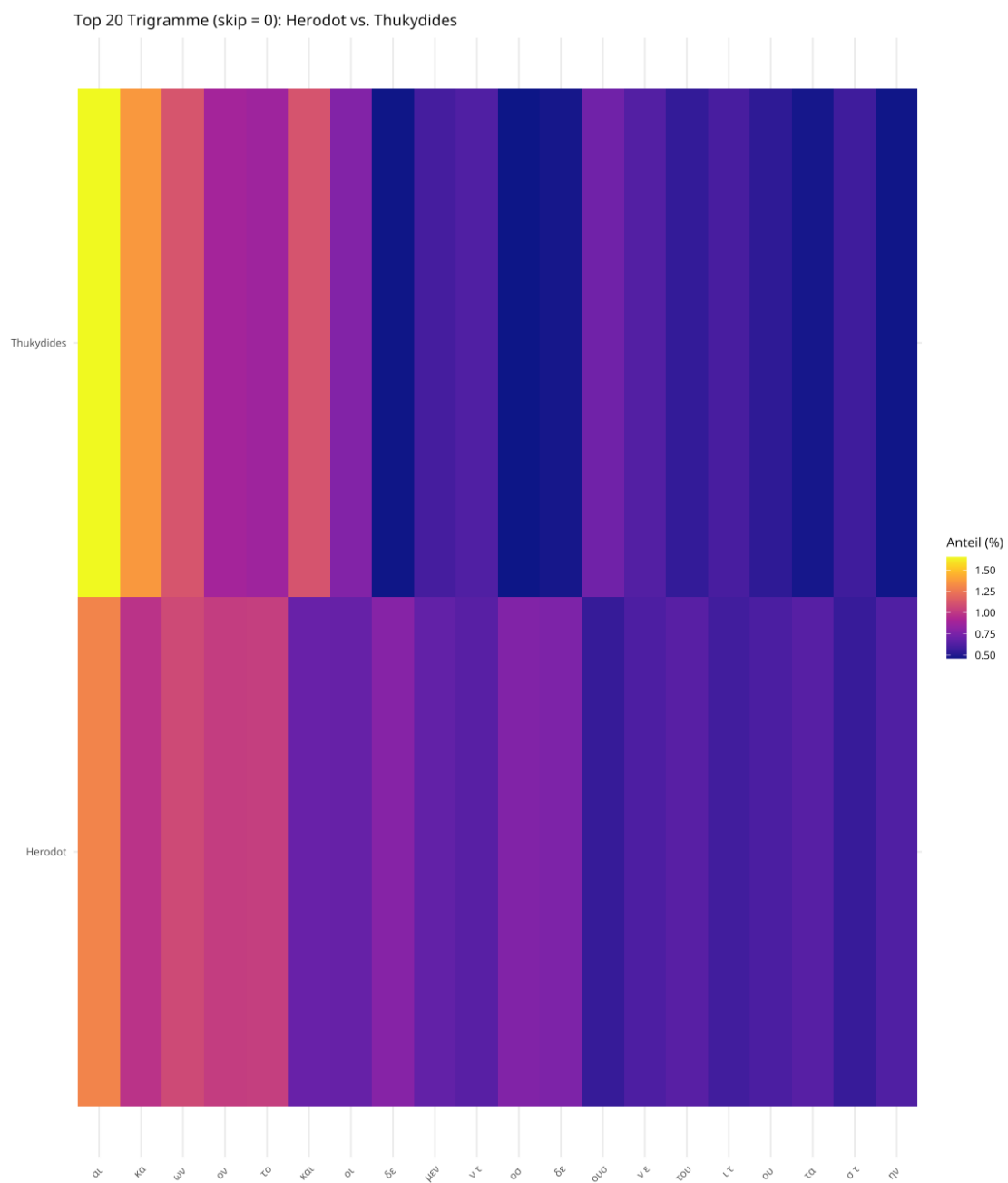
Top 50 Trigramme (skip = 0): Herodot vs. Thukydides

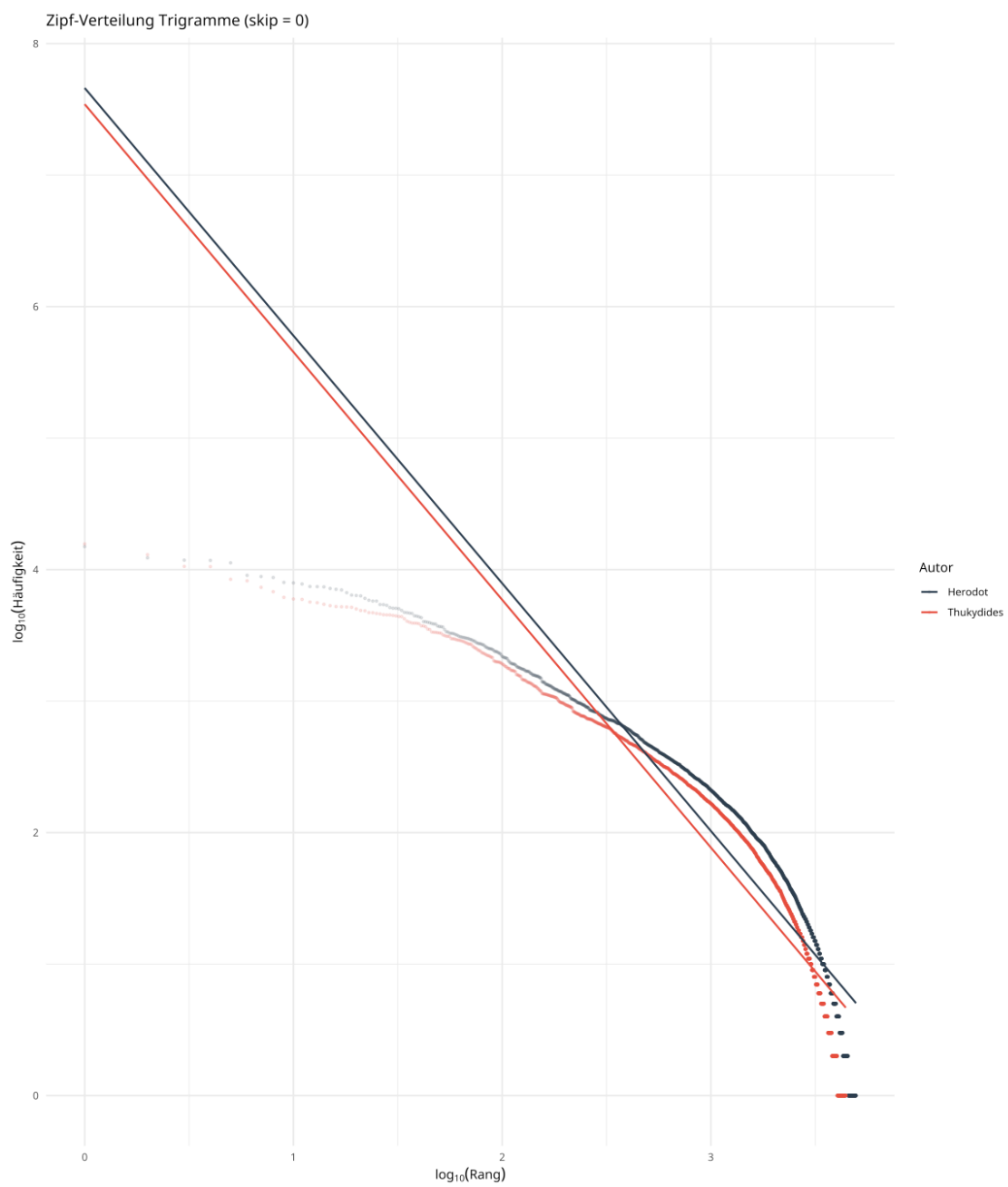


Top 50 Trigramme (skip = 0)

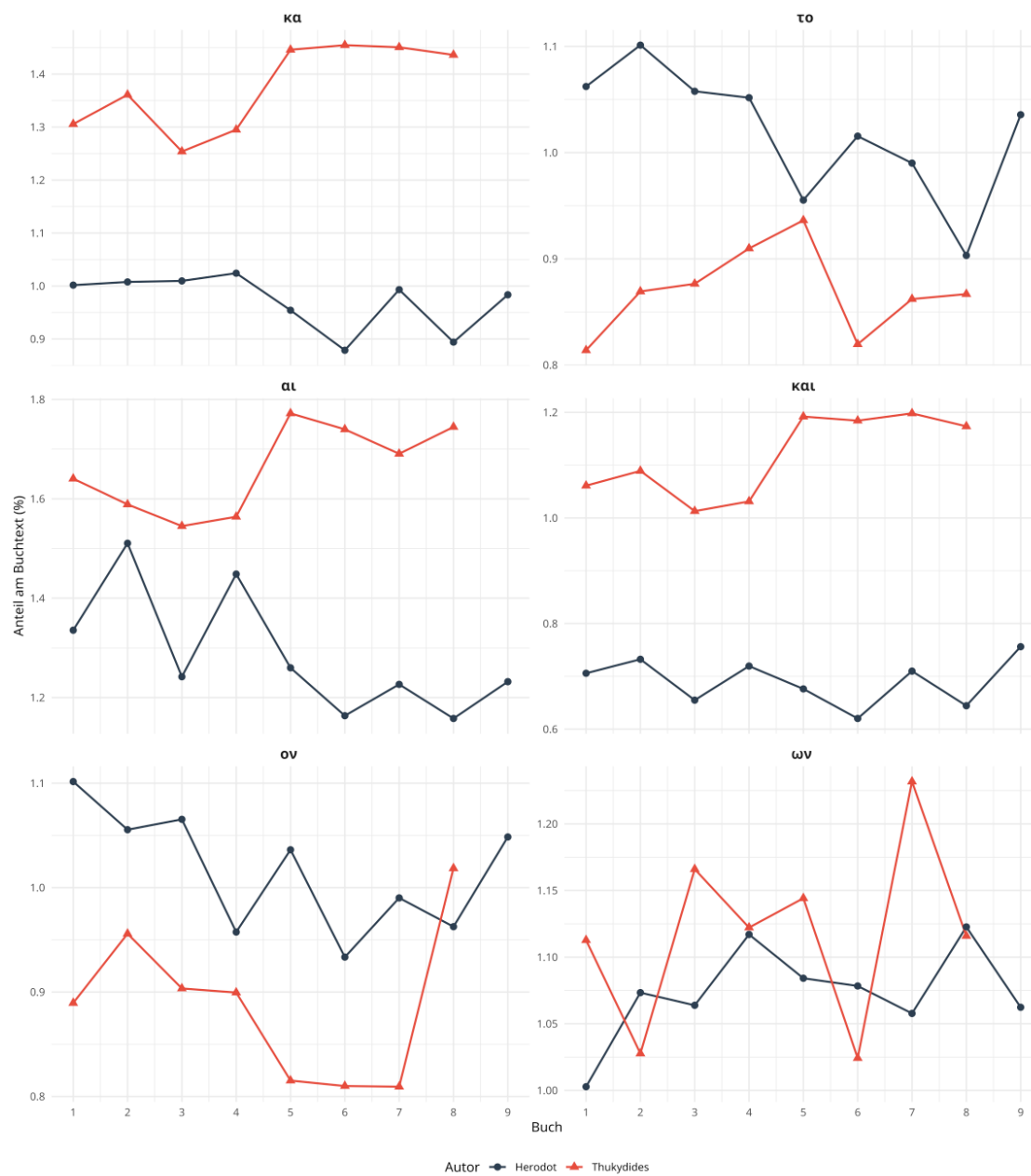




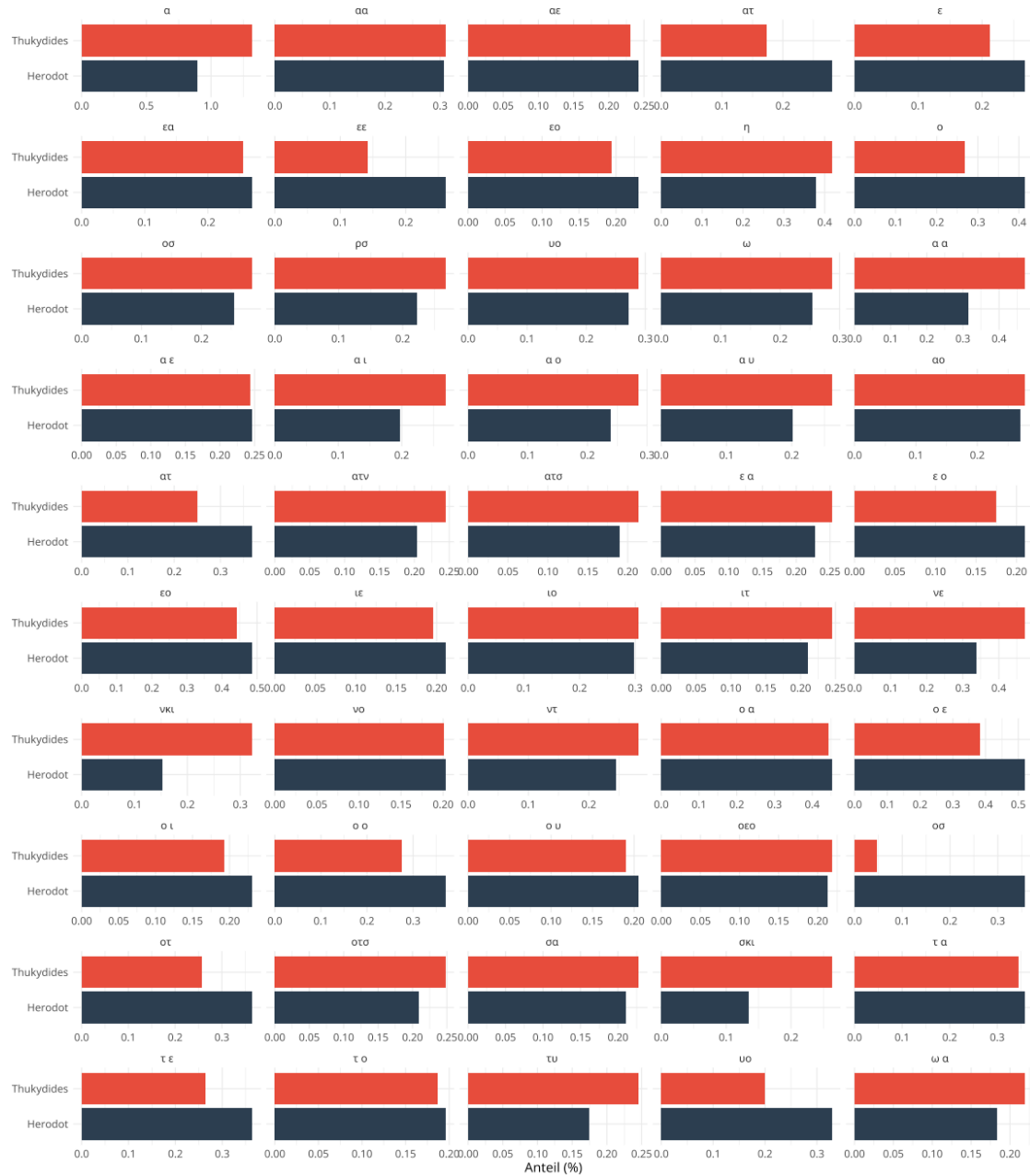




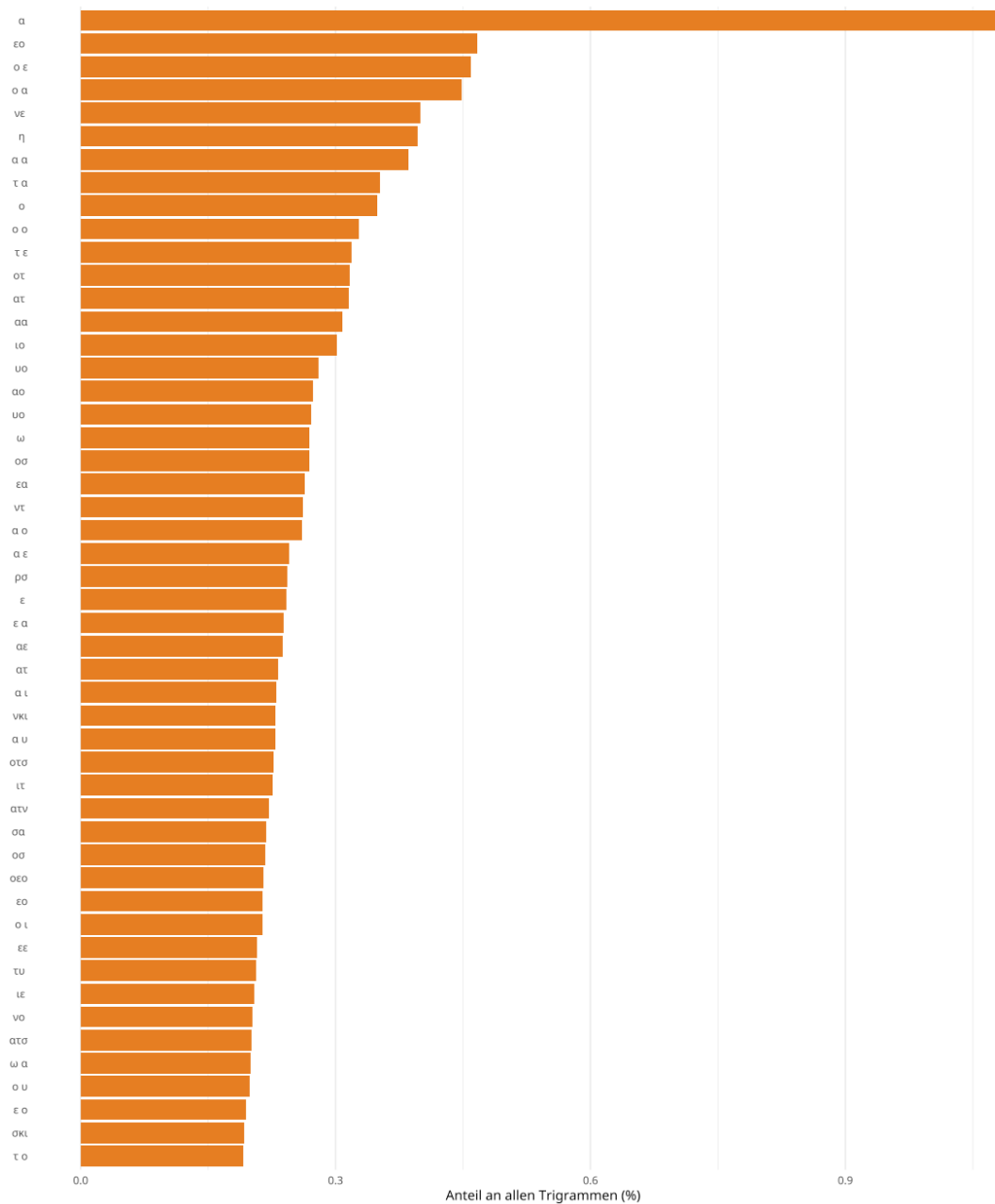
Ausgewählte Trigramme (skip = 0): Verlauf über die Bücher

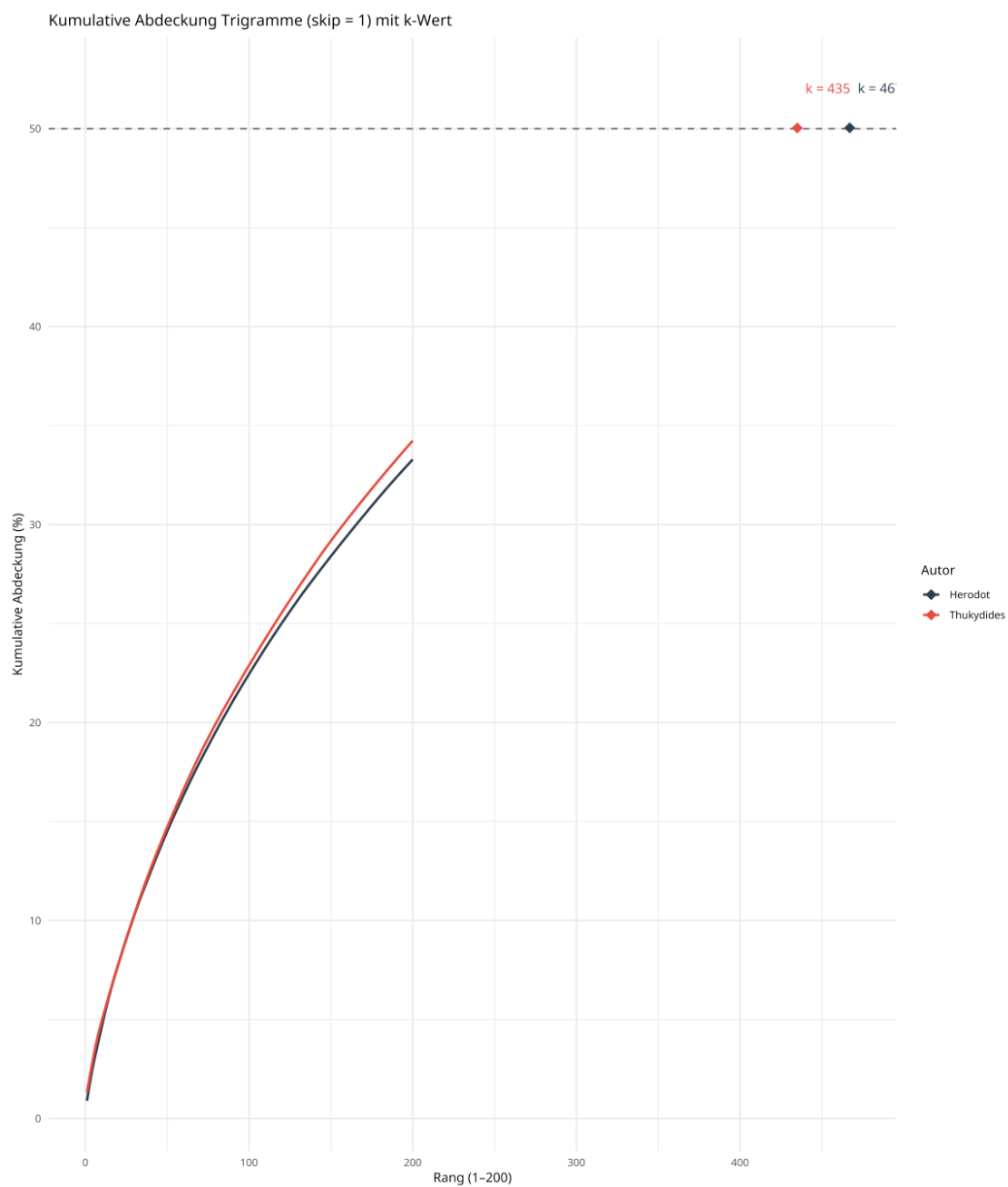


Top 50 Trigramme (skip = 1): Herodot vs. Thukydides

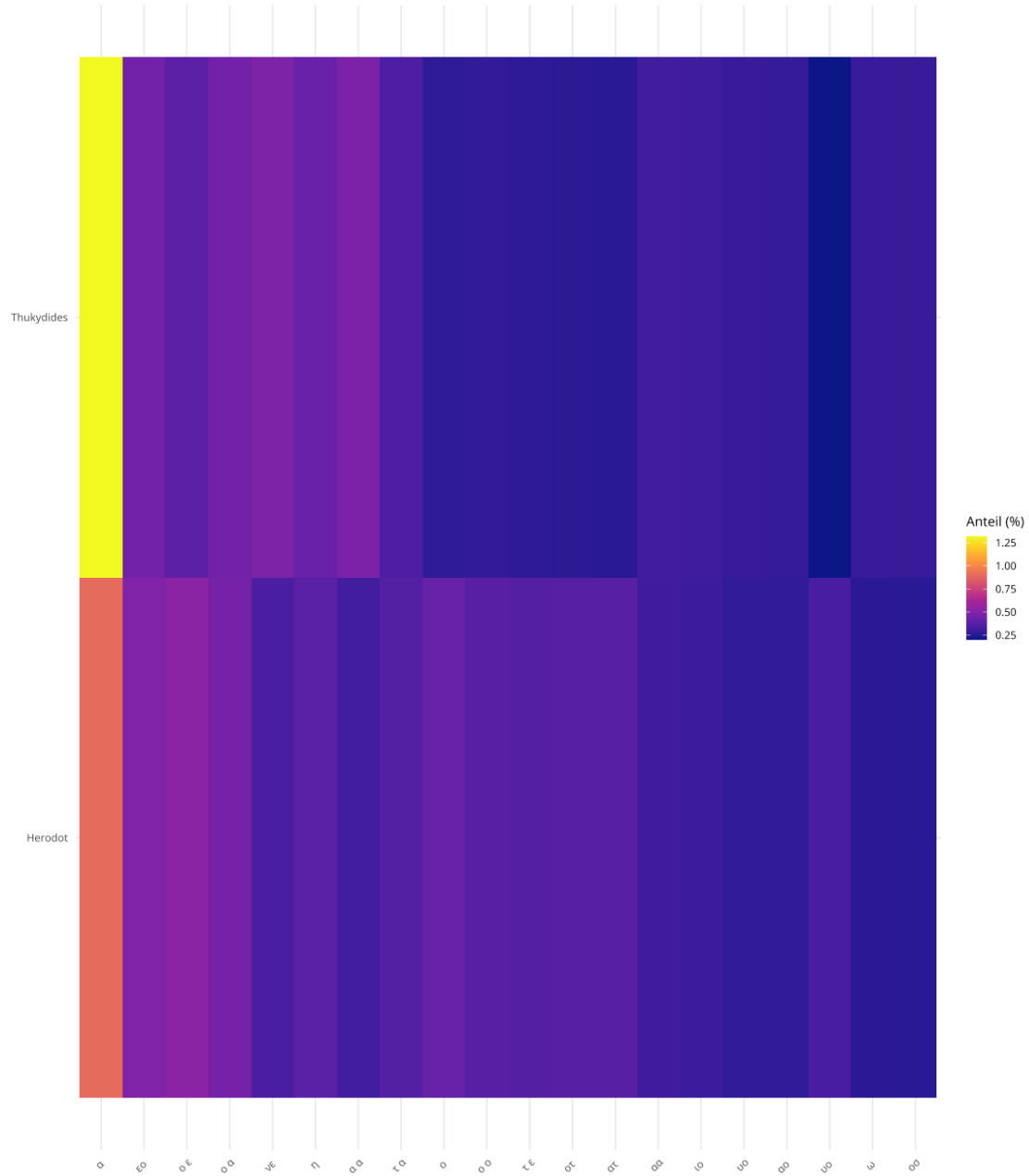


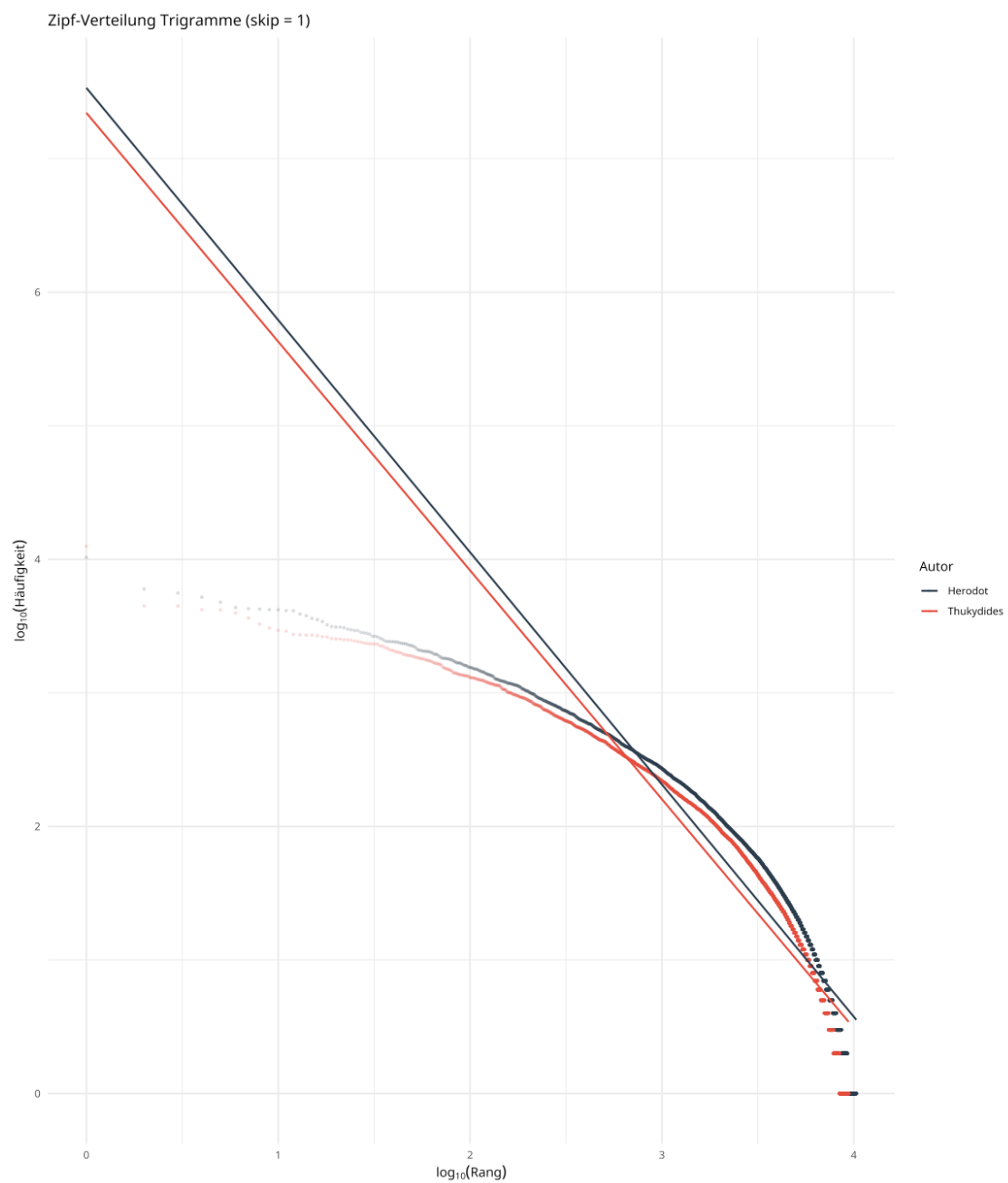
Top 50 Trigramme (skip = 1)

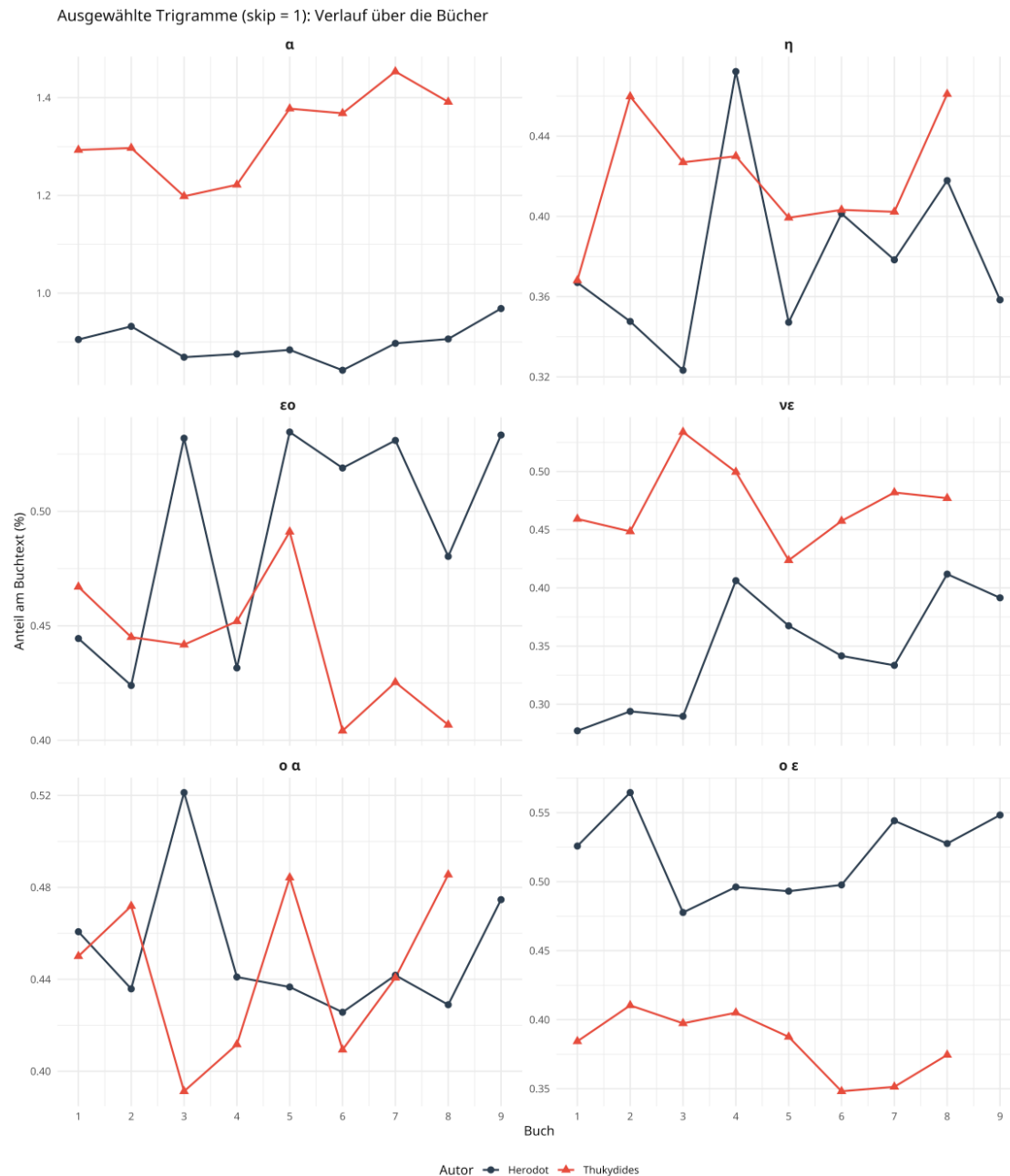




Top 20 Trigramme (skip = 1): Herodot vs. Thukydides







8. Vergleichsplots: skip = 0 vs. skip = 1

Diese Plots prüfen, ob die Autorenunterschiede über verschiedene Tokenisierungsparameter hinweg stabil sind.

C1. Kumulative Abdeckung: skip = 0 vs. skip = 1

linetype unterscheidet skip = 0 (durchgezogen) von skip = 1 (gestrichelt). Die Autorendifferenz bleibt bei beiden Varianten erhalten.

```

cumul_both <- map2(trigram_data, names(trigram_data), \(sd, nm) {
  sd$per_author |>
    group_by(author) |> arrange(desc(n), .by_group = TRUE) |>
    mutate(rank = row_number(),
           cumulative_pct = cumsum(n) / first(author_total) * 100) |>
    filter(rank <= 200) |> ungroup() |>
    mutate(skip = sprintf("skip = %d", sd$skip))
}) |> bind_rows()

p_cumul_compare <- ggplot(cumul_both,
                        aes(rank, cumulative_pct, color = author,
                           linetype = skip)) +
  geom_line(linewidth = 0.9) +
  geom_hline(yintercept = 50, linetype = "dashed", color = "grey40") +
  scale_color_manual(values = author_colors, name = "Autor") +
  scale_linetype_manual(values = c("skip = 0" = "solid", "skip = 1" =
"dashed"),
                        name = "Trigramm-Typ") +
  labs(x = "Rang (1-200)", y = "Kumulative Abdeckung (%)") +
  theme_greek()

p_cumul_compare
ggsave(here("Output_V8", "hdt_thuc_kumul_tri_skip_vergleich.png"),
       p_cumul_compare, width = 9, height = 5, dpi = 300)

```

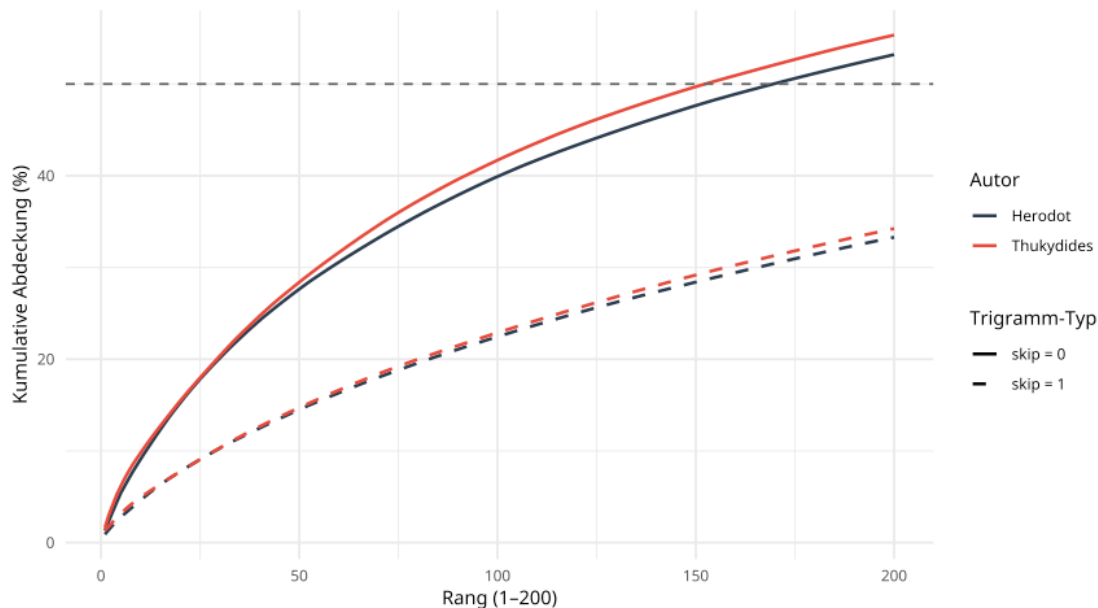


Figure 6

C2. Zipf-Exponenten: alle Ebenen

Gestrichelte Linie bei -1 = Idealwert. Wörter liegen nahe -1 , Trigramme bei ≈ -1.8 . Gleiche Exponenten auf derselben Ebene = gattungsbedingte Gemeinsamkeit.

```
zipf_all <- map(trigram_data, \(sd) {
  sd$per_author |>
  group_by(author) |> arrange(desc(n), .by_group = TRUE) |>
  mutate(rank = row_number()) |> ungroup() |>
  group_by(author) |>
  summarise(zipf_exponent = coef(lm(log10(n) ~ log10(rank)))[2],
    .groups = "drop") |>
  mutate(ebene = sprintf("Trigramme (skip = %d)", sd$skip))
}) |> bind_rows() |>
  bind_rows(zipf_exp_words |> mutate(ebene = "Wörter"))

zipf_all |>
  mutate(label = str_c(author, "\n", ebene)) |>
  ggplot(aes(zipf_exponent, label, fill = author)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = author_colors) +
  geom_vline(xintercept = -1, linetype = "dashed", color = "grey40") +
  labs(x = "Zipf-Exponent", y = NULL,
    subtitle = "Gestrichelte Linie: idealer Wert (-1)") +
  theme_greek()

ggsave(here("Output_V8", "hdt_thuc_zipf_alle_ebenen.png"),
  width = 8, height = 5, dpi = 300)
```

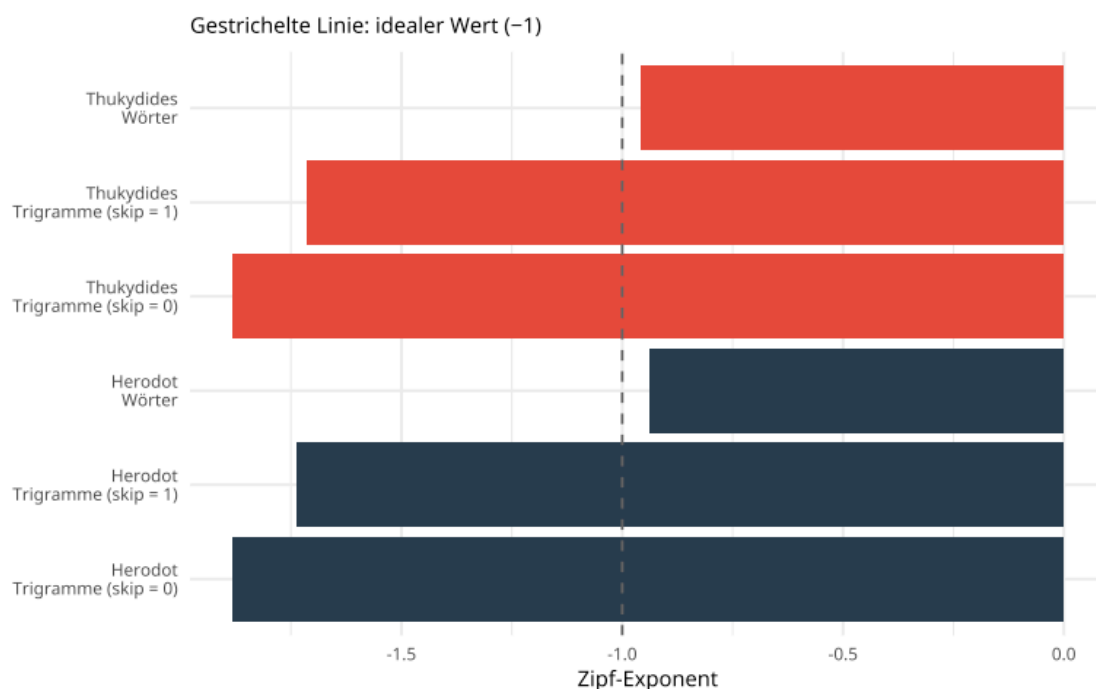


Figure 7

C3. k-Werte: alle Ebenen

Zentraler Ergebnisplot: Thukydides hat auf allen Ebenen den niedrigeren k-Wert.
`fct_reorder()` sortiert nach k-Wert, `geom_text()` zeigt die exakten Werte.

```
k_all <- map(trigram_data, \(sd) {
  sd$k_per_author |>
    mutate(ebene = sprintf("Trigramme (skip = %d)", sd$skip))
}) |> bind_rows() |>
  bind_rows(k_words_per_author |> mutate(ebene = "Wörter"))

k_all |>
  mutate(label = str_c(author, "\n", ebene)) |>
  ggplot(aes(k_abs, fct_reorder(label, k_abs), fill = author)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = k_abs), hjust = -0.2, size = 3.5,
            family = greek_font) +
  scale_fill_manual(values = author_colors) +
  labs(x = "k-Wert (absolut)", y = NULL) +
  theme_greek() +
  theme(panel.grid.major.y = element_blank())

ggsave(here("Output_V8", "hdt_thuc_kval_alle_ebenen.png"),
        width = 9, height = 5, dpi = 300)
```

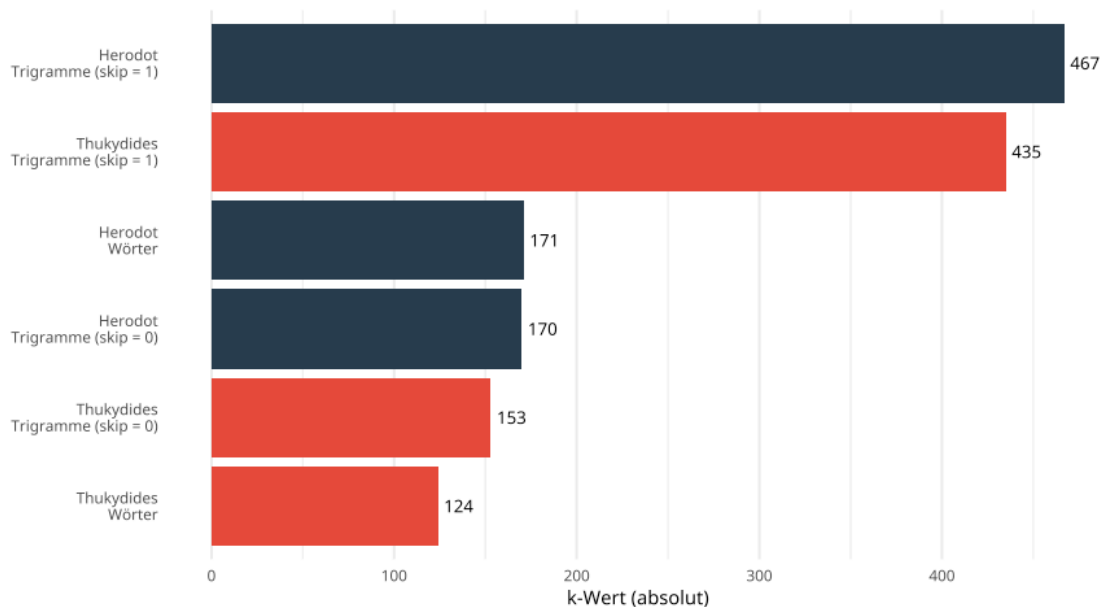


Figure 8

9. Zusammenfassung

```
cat("=== Tokenzahlen ===\n")
```

```

=== Tokenzahlen ===

cat("Wörter:\n")

Wörter:

print(all_tokens |> count(author, name = "woerter"))

# A tibble: 2 × 2
  author      woerter
  <chr>      <int>
1 Herodot    186407
2 Thukydides 150878

cat("\nTrigramme skip = 0:\n")

Trigramme skip = 0:

print(s0$tokens_autor)

# A tibble: 2 × 2
  author      token_count
  <chr>      <int>
1 Herodot    1150334
2 Thukydides  947511

cat("\nTrigramme skip = 1:\n")

Trigramme skip = 1:

print(s1$tokens_autor)

# A tibble: 2 × 2
  author      token_count
  <chr>      <int>
1 Herodot    1150316
2 Thukydides  947493

knitr::kable(zipf_all, digits = 3,
              col.names = c("Autor", "Zipf-Exponent", "Ebene"))

```

Table 1

Autor	Zipf-Exponent	Ebene
Herodot	-1.883	Trigramme (skip = 0)
Thukydides	-1.884	Trigramme (skip = 0)
Herodot	-1.739	Trigramme (skip = 1)
Thukydides	-1.713	Trigramme (skip = 1)
Herodot	-0.939	Wörter

Thukydides

-0.957

Wörter

```

cat("k_abs = absoluter Rang bei 50 % Abdeckung\n")
k_abs = absoluter Rang bei 50 % Abdeckung
cat("k_rel_types = k / Types (in %)\n")
k_rel_types = k / Types (in %)
cat("k_rel_tokens = k / Tokens (in ‰)\n\n")
k_rel_tokens = k / Tokens (in ‰)
k_all |>
  select(author, ebene, k_abs, k_rel_types, k_rel_tokens) |>
  knitr::kable(
    digits = c(0, 0, 0, 3, 5),
    col.names = c("Autor", "Ebene", "k (abs.)", "k / Types (%)", "k / Tokens
(‰)")
  )

```

Table 2

Autor	Ebene	k (abs.)	k / Types (%)	k / Tokens (‰)
Herodot	Trigramme (skip = 0)	170	3.428	0.14778
Thukydides	Trigramme (skip = 0)	153	3.455	0.16148
Herodot	Trigramme (skip = 1)	467	4.531	0.40598
Thukydides	Trigramme (skip = 1)	435	4.630	0.45911
Herodot	Wörter	171	0.610	0.91735
Thukydides	Wörter	124	0.569	0.82186